FINAL YEAR PROJECT / PROPOSAL

Smart Health Risk Predictor

An AI-Based Disease Risk Prediction System from Medical Reports

This is a web application that analyses the medical report uploaded by the user, predicts the risks for common diseases, and proposes further steps.

PROJECT DETAILS

SUBMITTED BY	Muhammad Abdullah
DEGREE PROGRAM	BS Information Technology
REGISTRATION NUMBER	F23BINFT1M01213
PROJECT SUPERVISOR	Sir Wahaj Ali
INSTITUTION	The Islamia University of Bahawalpur
SUBMISSION DATE	October 27, 2026

PROPOSED TECHNOLOGY

ASP.NET Core MVC • C# • Microsoft SQL Server • HTML / CSS / JavaScript • Python scikit-learn • ONNX Runtime

01 / PROJECT FRAMING

Abstract & Project Overview

A focused proposal for reading medical reports and estimating disease risk.

Abstract

The Smart Health Risk Predictor is an AI web application that aims to provide people with a preliminary assessment of the risk of developing certain diseases based on existing reports about their state of health. A user simply uploads a laboratory report in PDF format or as a picture. The application extracts the information from the document, identifies laboratory results, uses machine learning models, and displays the result together with some advice, without the user entering any data.

The prototype will support prediction of diabetes, heart disease, chronic kidney disease, liver disease, and stroke based on laboratory results and rule-based fever check using the recorded temperature. Breast cancer and Parkinson's disease, the predictors of which cannot be found in a routine laboratory report, can be evaluated using an additional optional manual input form. Results will be stored in the history only in case of a positive prediction.

The application will be built using ASP.NET Core MVC framework, C# programming language, and Microsoft SQL Server database. Models will be built using Python and scikit-learn on publicly available datasets and executed within the application using ONNX Runtime.

Project at a glance

ELEMENT	PROPOSAL
Primary users	People who have a medical report and want an early indication of their health risk.
Core purpose	Read laboratory reports automatically and estimate disease risk with suggestions.
AI support	Report reading (text extraction and OCR), disease risk models, and a confidence indication.
Result view	Risk level, probability, the values that were read, and suggestions for each disease.
Resolution boundary	Doctors, not the application, diagnose and treat. The result is a screening signal.

PROJECT PRINCIPLE AI outputs are screening aids. A doctor's examination and confirmatory tests remain essential.

02 / WHY THIS PROJECT

Problem, Aim & Objectives

The project turns the numbers in a medical report into a clear, early risk indication.

Problem statement

It often happens that people get the lab results but are unable to understand the figures, thus failing to notice the warning sign and going to see the doctor later. The existing web-based calculators generally consider a single disease at a time and require the user to input numerous technical values manually, which takes time and leads to mistakes.

The Smart Health Risk Predictor suggests a web-based solution that receives the lab report directly, extracts the values automatically and applies the machine learning algorithms to calculate the risks of developing several diseases. This is supposed to assist the user in making the decision about seeing the doctor sooner, although not substituting him with it. A software-based conclusion can signalize the issue, but only the doctor is able to diagnose a disease.

Project aim

AIM

The development and evaluation of an AI-powered web app that will read uploaded medical reports, assess the risk of developing common illnesses, and give advice, enabling users to determine earlier the right time to see a doctor.

Objectives

1. Create an interface for uploading a medical report (PDF and image formats) where a user can obtain the result without manual data entry.
2. Extract laboratory data from the uploaded file using PDF text recognition and OCR.
3. Build machine learning models to predict diabetes, heart disease, kidney disease, liver disease, and stroke using publicly available datasets.
4. Display the probability, level of risk, and amount of used data for each prediction to ensure the accuracy of the predictions.
5. Generate disease-related recommendations and store the results only if a disease is detected.

6. Conduct evaluation of the application using functional, performance tests, and user tasks.

Evaluation questions

- How well does the model predict the disease based on unseen data?
- How well does the system extract the values from reports with varying formats and units?
- Are the outcome and recommendations clear and useful to the user?

03 / SYSTEM BOUNDARIES

Scope, Users & Processing Flow

A clear separation between risk estimation and medical diagnosis.

Scope of the proposed system

The system would take as inputs lab reports in English language in the format of either PDF file containing text data or image (JPEG or PNG format). It would predict the risk of diabetes, heart disease, chronic kidney disease, liver disease, and stroke where there is sufficient information available in the lab report, and would also detect fever based on the temperature reading. The predictions related to breast cancer and Parkinson's disease would be available by filling out a separate optional manual input form.

Primary users

USER	ROLE IN THE WORKFLOW
General users	Upload a report, review the values that were read, and read the result and suggestions.
Doctors and health professionals	Outside the application. They examine the person, order confirmatory tests, diagnose, and treat.

Processing flow

01 UPLOAD The user chooses a report (PDF or photo).	**02 READ** The system extracts the text and finds the laboratory values.	**03 PREDICT** Models estimate the risks, and the result and suggestions are shown.

The application shows an estimate. A "disease predicted" result is a screening signal; it is not a diagnosis.

Outside the system boundary

- Diagnosis or treatment of patients or substituting the doctor's consultation and lab work.
- The guarantee of accuracy in every machine learning prediction.
- Scanned documents without the text layer, translations into other languages, and interpretation of the medical images including X-rays, CTs, and MRIs.

BOUNDARY

Smart Health Risk Predictor supports early awareness. Qualified doctors remain responsible for diagnosis and treatment.

04 / WHAT THE APPLICATION DOES

Functional Design & Result View

A practical workflow from an uploaded report to a clear result.

Core capabilities

CAPABILITY	PROPOSED BEHAVIOR
Report upload	Accept a laboratory report as a PDF (with text) or as a JPG or PNG photo.
Automatic reading	Extract text with PdfPig or Tesseract OCR, then find values using synonyms, plausible value ranges, and unit conversion.
Disease prediction	Run the trained model of each disease that has enough values in the report.
Confidence indication	Show how many values were found out of the total, with a High, Medium, or Low confidence label.
Suggestions	Show disease-specific advice such as follow-up tests and lifestyle guidance.
Fever check	Flag fever when the temperature in the report is 38 °C or higher (rule-based, not a model).
Result history	Save the result and the report file only when a disease is predicted.
Manual entry (optional)	Enter values by hand, mainly for breast cancer and Parkinson's disease.

Diseases covered

DISEASE	INPUT
Diabetes, heart disease, chronic kidney disease, liver disease, stroke	Read from the uploaded report (manual entry also possible)
Breast cancer, Parkinson's disease	Manual entry only (their inputs are not part of a routine laboratory report)
Fever	Temperature read from the report (rule-based)

Illustrative end-to-end flow

1. The user uploads his laboratory report on the main page.
2. The program recognizes the text and obtains the laboratory results.
3. For each disease with enough data, the ONNX model provides the probability.
4. On the result page, one can find the risk level, confidence, values recognized and recommendations.
5. In case of predicting any disease, the result and report will be saved in the history.

DESIGN RULE

A result based on few report values must be labeled low confidence, and a predicted disease must be presented as a screening signal with advice to consult a doctor.

System Architecture & Technology

The implementation uses the technologies listed below.

Technology stack

LAYER	TECHNOLOGY	ROLE
Presentation	HTML, CSS, JavaScript, Bootstrap 5	Upload page, result page, and responsive layout.
Application	ASP.NET Core MVC, C#	Routes, request handling, and business logic.
Data	Microsoft SQL Server, ADO.NET, stored procedures	Persistent storage of saved results.
Machine learning (training)	Python, scikit-learn, pandas	Model training and evaluation.
Machine learning (runtime)	ONNX Runtime	Runs the trained models inside the web application.
Report reading	PdfPig, Tesseract OCR	Text from PDF reports and from report photos.

Architecture at a glance

USER BROWSER	HTML • CSS • JavaScript: upload form and result page
APPLICATION SERVER	ASP.NET Core MVC: upload handling, controllers, business logic
REPORT READING	PdfPig and Tesseract: text and laboratory values
AI ANALYSIS	ONNX Runtime: disease risk probabilities
DATA	SQL Server: saved results; uploaded files kept outside the public folder

Data handled by the prototype

In the case of the saved results, the software saves the disease, probability, risk, source, date, input parameters, and file name. Results are saved only when a disease is detected by the software. The saved results are only accessible from the browser where the results were created, and files are not uploaded to the website folder.

STACK CONFIRMED	ASP.NET Core MVC + C# + SQL Server + scikit-learn + ONNX Runtime + HTML / CSS / JavaScript

Dataset & Machine-Learning Method

Evaluate each model against evidence, not assumptions.

Dataset preparation

Public medical datasets form the basis of training and testing. Each dataset is examined for missing values, impossible values, balancing classes, and appropriateness before any modeling can take place. The training and testing datasets are split before modeling begins, and the test cases are never used for training.

DISEASE	DATASET	RECORDS
Diabetes	Pima Indians Diabetes Database	768
Heart disease	Cleveland Heart Disease (UCI)	303
Breast cancer	Wisconsin Diagnostic Breast Cancer (UCI)	569
Chronic kidney disease	Chronic Kidney Disease (UCI)	400
Liver disease	Indian Liver Patient Dataset (UCI)	583
Parkinson's disease	Parkinson's Dataset (UCI)	195
Stroke	Stroke Prediction Dataset (Kaggle)	5,110

Proposed analysis pipeline

STAGE	METHOD / CHECK
1. Prepare	Clean each dataset, treat impossible zero values as missing, and document class balance and limits.
2. Split	80% training and 20% held-out test data, stratified by class.
3. Train	Train Logistic Regression, Random Forest, and SVM for each disease, with median imputation, scaling, and class weights.
4. Select	Choose the model with the best combined F1-score and ROC-AUC.
5. Export	Convert the selected model to ONNX and check that it gives the same predictions as the original.
6. Evaluate	Report accuracy, precision, recall, F1-score, ROC-AUC, and the confusion matrix, and state the limits.

Preliminary results (first training run)

DISEASE	MODEL	ACCURACY	PRECISION	RECALL	F1	ROC-AUC
Diabetes	SVM	72.7%	59.1%	72.2%	65.0%	0.81

Heart disease	Logistic Regression	85.2%	85.2%	82.1%	83.6%	0.92
Breast cancer	SVM	95.6%	93.0%	95.2%	94.1%	0.99
Kidney disease	Random Forest	100%	100%	100%	100%	1.00
Liver disease	Random Forest	73.5%	75.5%	92.8%	83.2%	0.79
Parkinson's disease	Random Forest	92.3%	93.3%	96.6%	94.9%	0.98
Stroke	Logistic Regression	74.6%	13.8%	80.0%	23.5%	0.84

These figures represent an initial analysis which will be reviewed later on in this project. The kidney data set is small in size and is well-separated with 80 test instances, therefore 100% accuracy for this data set cannot be considered realistic. There are very few positives in the stroke data set; hence, the system detects all positives while giving many false positives.

Human review safeguards

Output from the model is an advisory recommendation. In the results page, the values read from the report are displayed for comparison with the original value, the amount of data used to make each of the predictions is also displayed, and there is always an advisory to seek a doctor's opinion.

EVIDENCE FIRST

Public datasets allow evaluation, but they do not guarantee strong performance on real reports. Results must be measured and reported honestly.

07 / DELIVERY & ASSESSMENT

Testing, Risks & Expected Outcome

A realistic prototype with clear success criteria and boundaries.

Testing plan

AREA	CHECK
Application functions	Upload PDF and image reports, view the result, save only when a disease is predicted, and open the saved history.
Report reading	Test sample reports with different layouts and units, and count how many values are read correctly.
Model quality	Evaluate on held-out data with the stated metrics, and compare the ONNX output with the original model.
Confidence handling	Check that reports with few values receive a lower confidence label.

Usability	Ask representative users to upload a report and explain the result and the suggestions.

Key risks and mitigation

RISK	MITIGATION
Report layouts differ and values are misread	Use synonyms, value ranges, and unit rules; show the values read; ask for a clear image.
Small or imbalanced datasets	Use class weights, show a confidence label, and disclose the limits.
Reports do not contain every input a model needs	Predict only with a minimum number of values, label the confidence, and offer manual entry.
A result is mistaken for a diagnosis	Show clear disclaimers and advise consulting a doctor.
Privacy of medical files	Store files outside the public folder; show them only to the browser that uploaded them.

Expected outcome

PROJECT SUMMARY	Read the report. Estimate the risk. Suggest next steps. Leave diagnosis to doctors.

The expected deliverable will be a working web prototype developed using the ASP.NET Core MVC framework, C# language, SQL Server, scikit-learn and ONNX Runtime. A user will be able to input a lab report and get instantaneous results including the risk estimates, values read, and recommendations. In addition, the deliverables will include the trained models, training scripts, user manual, and test report.

Conclusion

The Smart Health Risk Predictor utilizes machine learning to solve a problem of everyday life. The usefulness of this product is not its guarantee of diagnosing any disease but its organized way of analyzing a report, making a risk assessment, and directing a patient towards the right decision.